

$$\theta_0^{(2)} = 5.348, \quad \theta_1^{(2)} = 16.492$$

So after two iterations, the model is:

$$h_\theta(x) = 5.348 + 16.492x$$

That is a valid two-iteration simulation. But for the full problem, you should continue for more iterations and show the values in a table until the cost becomes stable.

Part 3: Find the answers to these conceptual questions

1. Explain the principle of gradient descent. What are the roles of the learning rate and the number of iterations in this algorithm?
2. What happens if the learning rate is too large?
3. What happens if the learning rate is too small?
4. Explain the differences between gradient descent, stochastic gradient descent (SGD), and mini-batch gradient descent.
5. What does it mean for gradient descent to converge? How can you tell if gradient descent has failed to converge in practice?
6. What are some potential drawbacks or challenges of using stochastic gradient descent, especially in terms of the final stages of convergence?
7. Describe the cost function used in linear regression. Why is it important to minimize this function?
8. What is the hypothesis function in linear regression? How does it differ in its formulation between simple linear regression and multiple linear regression?

please find answers of these conceptual questions.

Here are simple exam-style answers for all 8 conceptual questions.

1. Explain the principle of gradient descent. What are the roles of learning rate and number of iterations?

Gradient descent is an optimization algorithm used to minimize the cost function.

In linear regression, we start with initial values of parameters, such as:

$$\theta_0, \theta_1$$

Then we repeatedly update them in the direction that reduces the cost.

The update rule is:

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

where:

- θ_j is a parameter
- α is the learning rate
- $\frac{\partial J}{\partial \theta_j}$ is the gradient of the cost function

The **learning rate** controls how large each step is.

The **number of iterations** controls how many times the parameters are updated.

If the number of iterations is too small, the algorithm may stop before reaching the minimum. If it is large enough, the algorithm can get closer to the minimum cost.

2. What happens if the learning rate is too large?

If the learning rate is too large, gradient descent may take very big steps.

As a result:

- it may overshoot the minimum,
- the cost may increase instead of decrease,
- the values may oscillate,
- the algorithm may fail to converge.

In simple words:

A very large learning rate can make gradient descent unstable.

3. What happens if the learning rate is too small?

If the learning rate is too small, gradient descent takes very tiny steps.

As a result:

- convergence becomes very slow,
- many iterations are needed,
- training takes more time,
- the algorithm may appear stuck even though it is moving slowly.

In simple words:

A very small learning rate is safe but slow.

4. Explain the differences between gradient descent, stochastic gradient descent, and mini-batch gradient descent.

Batch gradient descent

Batch gradient descent uses the **entire training dataset** to calculate the gradient before each update.

It is stable but can be slow for large datasets.

Stochastic gradient descent, SGD

Stochastic gradient descent uses **one training example at a time** to calculate the gradient and update the parameters.

It is faster, but the updates are noisy.

Mini-batch gradient descent

Mini-batch gradient descent uses a **small group of training examples** at a time.

It is a balance between batch gradient descent and stochastic gradient descent.

Method	Data used per update	Advantage	Disadvantage
Batch gradient descent	Whole dataset	Stable updates	Slow for large data
SGD	One example	Fast updates	Noisy, unstable path
Mini-batch gradient descent	Small batch	Fast and fairly stable	Batch size must be chosen

5. What does it mean for gradient descent to converge? How can you tell if gradient descent has failed to converge

in practice?

Gradient descent **converges** when the cost function becomes very small or stops changing significantly.

That means the parameters have reached values close to the minimum of the cost function.

You can tell gradient descent has converged when:

- the cost decreases and becomes almost constant,
- parameter values stop changing much,
- the model predictions stop improving significantly.

Gradient descent has failed to converge if:

- the cost keeps increasing,
- the cost keeps jumping up and down,
- the parameters become extremely large,
- the algorithm does not reach a stable minimum.

In practice, we often plot the cost J against the number of iterations. If the curve decreases smoothly and flattens, it is converging. If it increases or oscillates strongly, it is not converging.

6. What are some potential drawbacks or challenges of using stochastic gradient descent, especially in terms of the final stages of convergence?

Stochastic gradient descent updates the parameters using only one training example at a time.

Because of this, the updates are noisy.

The main challenges are:

- the cost function may keep fluctuating,
- it may not smoothly reach the exact minimum,
- it may keep moving around the minimum,
- choosing a good learning rate is difficult,
- it may need a decaying learning rate to settle near the minimum.

In the final stages, SGD may not converge exactly to the minimum because each individual data point gives a slightly different update direction.

So SGD is fast, but less stable than batch gradient descent.

7. Describe the cost function used in linear regression. Why is it important to minimize this function?

In linear regression, the commonly used cost function is the **Mean Squared Error cost function**.

It is:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

where:

- m is the number of training examples,
- $h_{\theta}(x^{(i)})$ is the predicted value,
- $y^{(i)}$ is the actual value,
- $h_{\theta}(x^{(i)}) - y^{(i)}$ is the error.

This function measures how far the predictions are from the actual values.

We minimize this function because smaller cost means smaller prediction error.

In simple words:

Minimizing the cost function helps us find the best-fitting line for the data.

The factor $\frac{1}{2m}$ is used to average the squared error and make differentiation easier.

8. What is the hypothesis function in linear regression? How does it differ between simple linear regression and multiple linear regression?

The **hypothesis function** is the prediction function used by the model.

It gives the predicted output for a given input.

Simple linear regression

Simple linear regression has only **one input feature**.

The hypothesis is:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Example:

$$\text{score} = \theta_0 + \theta_1(\text{hours studied})$$

Multiple linear regression

Multiple linear regression has **more than one input feature**.

The hypothesis is:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Example:

$$\text{house price} = \theta_0 + \theta_1(\text{size}) + \theta_2(\text{bedrooms}) + \theta_3(\text{location score})$$

So the difference is:

Type	Number of features	Hypothesis
Simple linear regression	One feature	$h_{\theta}(x) = \theta_0 + \theta_1 x$
Multiple linear regression	Multiple features	$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$

Final idea:

Simple linear regression predicts using one variable, while multiple linear regression predicts using several variables.